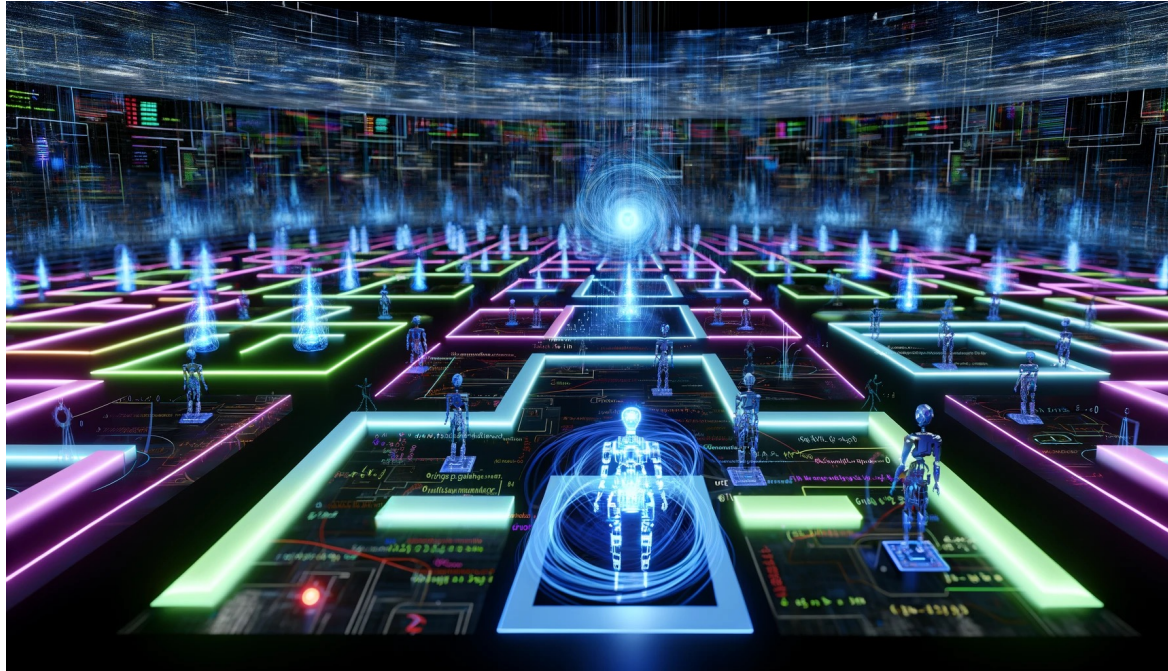


16-831: Introduction to Robot Learning, Spring 2024

Introduction to Reinforcement Learning



Generated by GPT-4 with DALL·E

Instructor: Guanya Shi

Recording Reminder

- ☐ This session will be audio recorded for educational use by other students in this course.

Course Structure

Latest schedule: <https://16-831-s24.github.io/lectures>

- ☐ Topic group 1: Course introduction: What is robot learning and why?
- ☐ Topic group 2: Machine/Deep learning refresher
- ☐ Topic group 3: Imitation learning
- ☒ Topic group 4: Model-free reinforcement learning
- ☐ Topic group 5: Model-based reinforcement learning (one guest lecture)
- ☐ Topic group 6: Reinforcement learning from offline data (one guest lecture)
- ☐ Topic group 7: Bandit, preference-based learning, exploration
- ☐ Topic group 8: Specialized topics (one guest lecture)
- ☐ Project presentation & poster session

Outline

(Some blackboard today!)

- ❑ Why RL?
- ❑ Definitions: MDP, POMDP, return, value functions

Tabular case:

- ❑ Policy evaluation
- ❑ Policy improvement
 - Policy iteration
 - Value iteration

(Some slides are from 22F, 23F, Berkeley CS 285, and CMU 10-703)

Outline

(Some blackboard today!)

❑ Why RL?

❑ Definitions: MDP, POMDP, return, value functions

Tabular case:

❑ Policy evaluation

❑ Policy improvement

- Policy iteration
- Value iteration

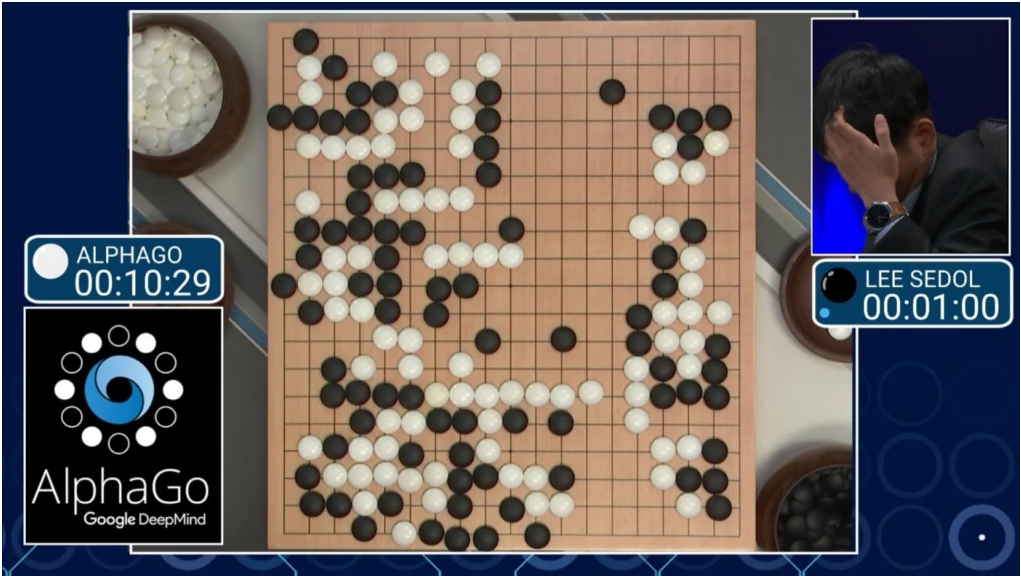
(Some slides are from 22F, 23F, Berkeley CS 285, and CMU 10-703)

Limitations of Imitation Learning

- ❑ Need experts (e.g., human)
 - Could be expensive
- ❑ Can not go beyond the expert level
 - Not enough for “superintelligence”
- ❑ Ideally, AI should be able to learn autonomously!

RL Success

❑ AlphaGo, 2016



Summary [\[edit \]](#)

Game	Date	Black	White	Result	Moves
1	9 March 2016	Lee Sedol	AlphaGo	Lee Sedol resigned	186 Game 1
2	10 March 2016	AlphaGo	Lee Sedol	Lee Sedol resigned	211 Game 2
3	12 March 2016	Lee Sedol	AlphaGo	Lee Sedol resigned	176 Game 3
4	13 March 2016	AlphaGo	Lee Sedol	AlphaGo resigned	180 Game 4
5	15 March 2016	Lee Sedol ^{[note 1]}	AlphaGo	Lee Sedol resigned	280 Game 5
Result: AlphaGo 4 – 1 Lee Sedol					

RL Success

□ AlphaZero: chess, go, shogi

Configuration and strength^[62]

Versions	Hardware	Elo rating	Date	Results
AlphaGo Fan	176 GPUs, ^[53] distributed	3,144 ^[52]	Oct 2015	5:0 against Fan Hui
AlphaGo Lee	48 TPUs, ^[53] distributed	3,739 ^[52]	Mar 2016	4:1 against Lee Sedol
AlphaGo Master	4 TPUs, ^[53] single machine	4,858 ^[52]	May 2017	60:0 against professional players; Future of Go Summit
AlphaGo Zero (40 block)	4 TPUs, ^[53] single machine	5,185 ^[52]	Oct 2017	100:0 against AlphaGo Lee 89:11 against AlphaGo Master
AlphaZero (20 block)	4 TPUs, single machine	5,018 ^[63]	Dec 2017	60:40 against AlphaGo Zero (20 block)

RL Success

- ❑ AlphaStar: Grandmaster level in StarCraft II



RL Success

- ❑ Jumping, 400m dash of Cassie (biped)



RL Success

- ❑ Legged locomotion in challenging terrains using egocentric vision



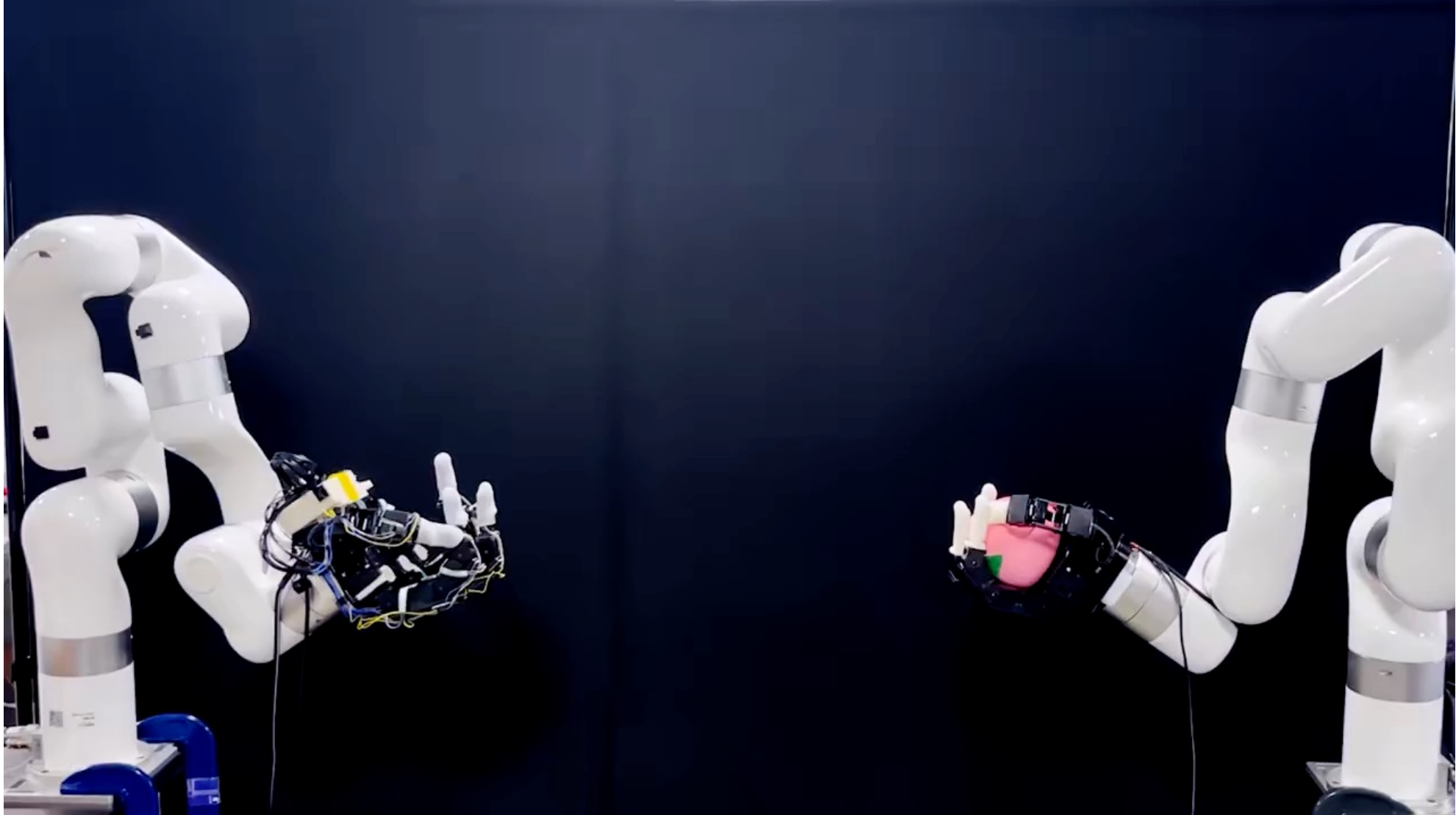
RL Success

- Agile but safe: learning collision-free high-speed legged locomotion



RL Success

- ❑ Dynamic Handover: Throw and Catch with Bimanual Hands



RL Success

- ❑ Champion-level drone racing using deep reinforcement learning



Outline

(Some blackboard today!)

□ Why RL?

□ Definitions: MDP, POMDP, return, value functions

Tabular case:

□ Policy evaluation

□ Policy improvement

- Policy iteration
- Value iteration

(Some slides are from 22F, 23F, Berkeley CS 285, and CMU 10-703)

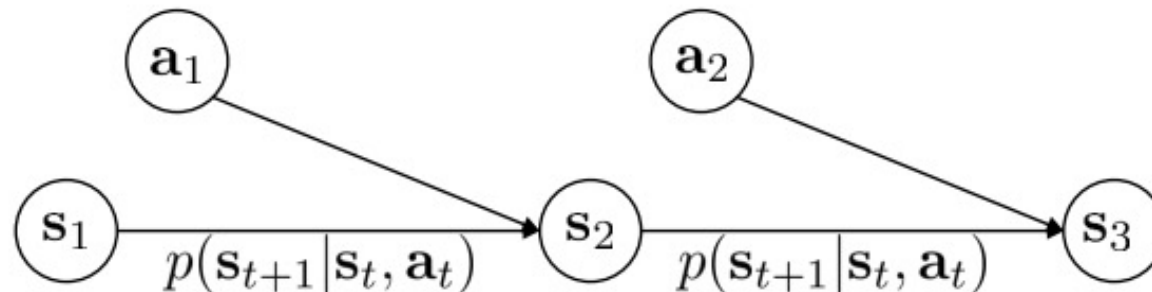
Recap: MDP

□ Definitions:

- S is the state space. $s_t \in S$ is the state at time step t .
- A is the action space. $a_t \in A$ is the action at time step t .
- O is the observation space. $o_t \in O$ is the observation at time step t .
- p is the one-step transition probability (a.k.a., dynamics): $s_{t+1} \sim p(\cdot | s_t, a_t)$
- h is the observation model: $o_t \sim h(\cdot | s_t)$
- $r: S \times A \rightarrow \mathbb{R}$ is the reward function.

□ A Markov decision process (MDP) is a tuple (S, A, p, r)

- Goal: learn a policy $\pi_\theta(a_t | s_t)$



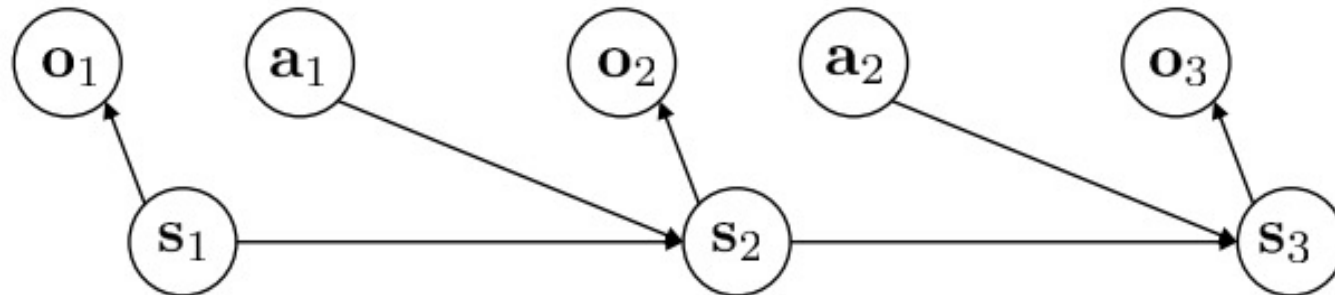
Recap: POMDP

□ Definitions:

- S is the state space. $s_t \in S$ is the state at time step t .
- A is the action space. $a_t \in A$ is the action at time step t .
- O is the observation space. $o_t \in O$ is the observation at time step t .
- p is the one-step transition probability (a.k.a., dynamics): $s_{t+1} \sim p(\cdot | s_t, a_t)$
- h is the observation model: $o_t \sim h(\cdot | s_t)$
- $r: S \times A \rightarrow \mathbb{R}$ is the reward function.

□ A partially observed Markov decision process (POMDP) is a tuple (S, A, O, p, h, r)

- Goal: learn a policy $\pi_\theta(a_t | o_t)$



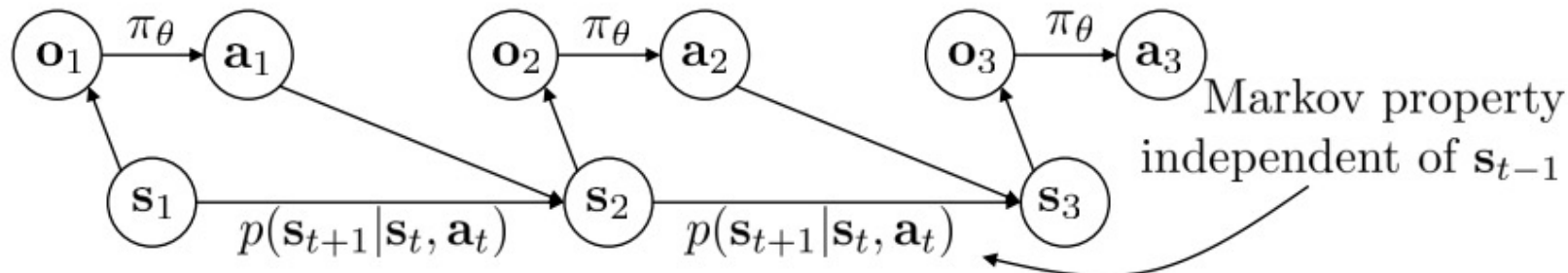
Recap: Markov Property

□ Definitions:

- S is the state space. $s_t \in S$ is the state at time step t .
- A is the action space. $a_t \in A$ is the action at time step t .
- O is the observation space. $o_t \in O$ is the observation at time step t .
- p is the one-step transition probability (a.k.a., dynamics): $s_{t+1} \sim p(\cdot | s_t, a_t)$
- h is the observation model: $o_t \sim h(\cdot | s_t)$
- $r: S \times A \rightarrow \mathbb{R}$ is the reward function.

□ Why Markov?

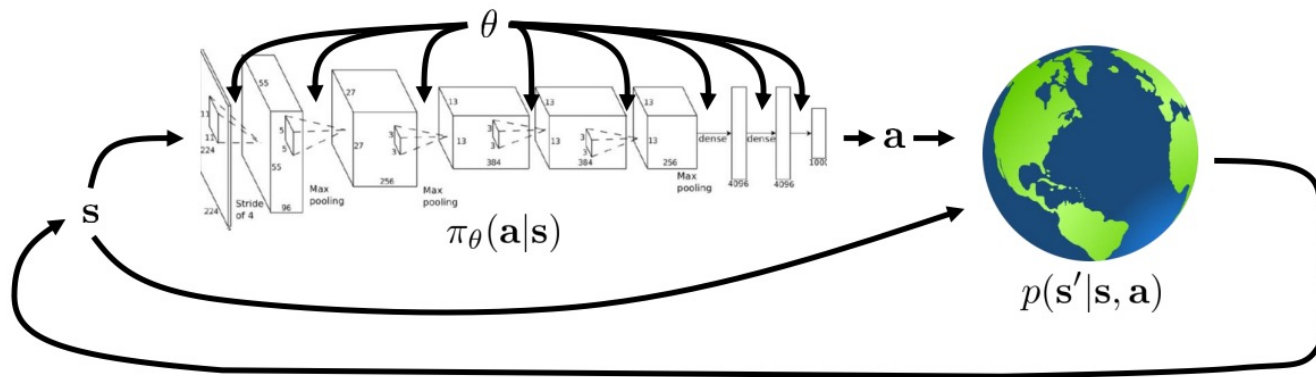
- Allow us to throw away history once state is known!



Andrey Markov

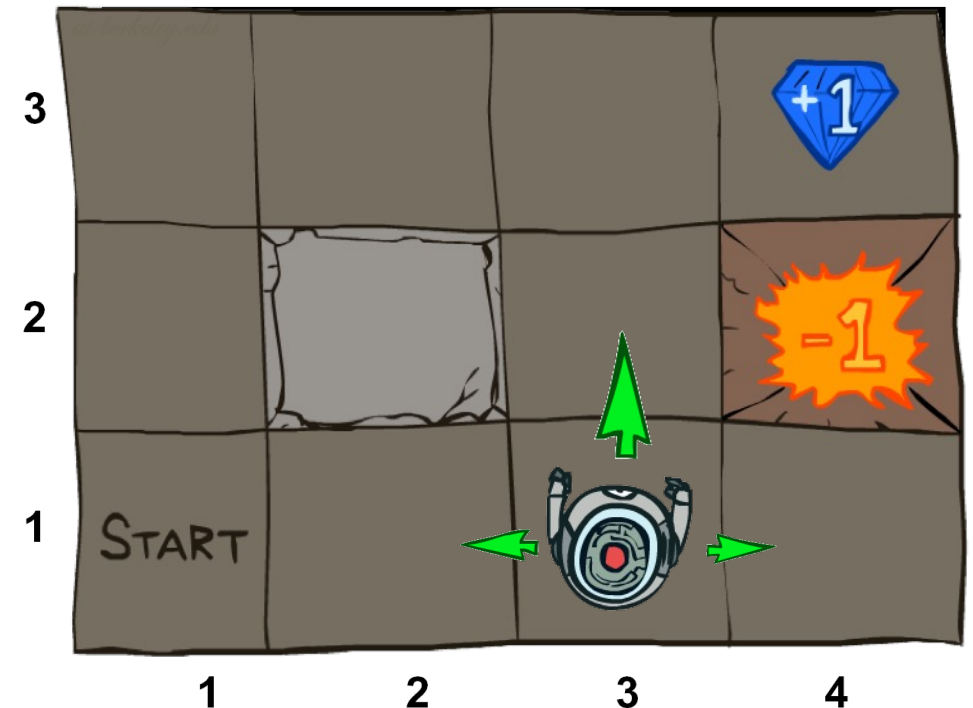
Recap: The Goal of RL

- ❑ Finite horizon case: T is finite
- ❑ Infinite horizon case: $T = \infty$
- ❑ The cumulative reward is often **discounted**: $\sum_t \gamma^t r(s_t, a_t)$ where $0 < \gamma \leq 1$
- ❑ Goal: find a policy to **maximize the cumulative reward**
 - Example: grid world



$$\underbrace{p_\theta(s_1, a_1, \dots, s_T, a_T)}_{\pi_\theta(\tau)} = p(s_1) \prod_{t=1}^T \pi_\theta(a_t | s_t) p(s_{t+1} | s_t, a_t)$$

$$\theta^* = \arg \max_{\theta} E_{\tau \sim p_\theta(\tau)} \left[\sum_t r(s_t, a_t) \right]$$



Returns G_t

- Sum of future rewards

$$G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} \dots = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$$

- Episodic setting (finite horizon):

$$G_t = r_{t+1} + r_{t+2} + \dots + r_T$$

- The action sequence is $s_0, a_0, r_1, s_1, a_1, r_2, \dots$
 - Some books/papers might use different index
- We will focus on the discounted infinite horizon case today

Value Functions

- ❑ State-value function (V): the expected return starting from state s , and then following policy π

$$V^\pi(s) = \mathbb{E}_\pi[G_0 | s_0 = s]$$

- ❑ Action-value function (Q): the expected return starting from state s , taking action a , and then following policy π

$$Q^\pi(s, a) = \mathbb{E}_\pi[G_0 | s_0 = s, a_0 = a]$$

- ❑ Question: why do we need both V and Q? Why does $Q^\pi(s, a) > V^\pi(s)$ mean?

Optimal Value Functions

□ Optimal V:

$$V^*(s) = \max_{\pi} V^{\pi}(s) = \max_{\pi} \mathbb{E}_{\pi}[G_0 | s_0 = s]$$

□ Optimal Q:

$$Q^*(s, a) = \max_{\pi} Q^{\pi}(s, a) = \max_{\pi} \mathbb{E}_{\pi}[G_0 | s_0 = s, a_0 = a]$$

□ Optimal policy:

$$\pi^*(s) = \operatorname{argmax}_{\pi} V^{\pi}(s)$$

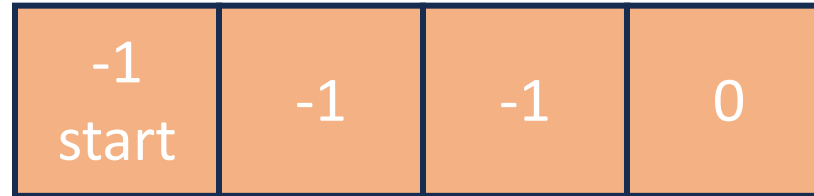
□ Properties:

$$V^*(s) = \max_a Q^*(s, a)$$

$$\pi^*(s) = \operatorname{argmax}_a Q^*(s, a)$$

- If we know $Q^*(s, a)$, we immediately have the optimal policy! Do not need the dynamics
- If we only know $V^*(s)$, we need the dynamics to do one step lookahead to choose the optimal action

Example: 1-d Grid World



- ❑ Four states: a (start), b, c, d
- ❑ Two actions: left, right, stay
- ❑ Assume no discount and deterministic transitions
- ❑ $V^*(a) = -3$, $V^*(b) = -2$, $V^*(c) = -1$, $V^*(d) = 0$
- ❑ $Q^*(a, \text{stay}) = -4$, $Q^*(a, \text{right}) = -3$, so $\pi^*(a) = \text{right}$

Outline

(Some blackboard today!)

☐ Why RL?

☐ Definitions: MDP, POMDP, return, value functions

Tabular case:

☒ Policy evaluation

☐ Policy improvement

- Policy iteration
- Value iteration

(Some slides are from 22F, 23F, Berkeley CS 285, and CMU 10-703)

Policy Evaluation

- Goal: given some π (**might not be optimal**), compute its value functions

$$V^\pi(s) = \mathbb{E}_\pi[G_0 | s_0 = s]$$

$$Q^\pi(s, a) = \mathbb{E}_\pi[G_0 | s_0 = s, a_0 = a]$$

- Recursive relationships for returns:

- One-step reward + discounted next step's return!

$$\begin{aligned} G_t &= r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} \dots = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1} \\ &= r_{t+1} + \gamma(r_{t+2} + \gamma r_{t+3} \dots) = r_{t+1} + \gamma G_{t+1} \end{aligned}$$

Policy Evaluation

- Goal: given some π (might not be optimal), compute its value functions

$$V^\pi(s) = \mathbb{E}_\pi[G_0 | s_0 = s]$$

$$Q^\pi(s, a) = \mathbb{E}_\pi[G_0 | s_0 = s, a_0 = a]$$

- Leveraging the recursive relationship:

$$\begin{aligned} V^\pi(s) &= \mathbb{E}_\pi[G_0 | s_0 = s] \\ &= \mathbb{E}_\pi[r_1 + \gamma G_1 | s_0 = s] \\ &= \sum_a \pi(a|s) \sum_{s', r} p(s', r | s, a) [r + \gamma \cdot V^\pi(s')] \end{aligned}$$

- Simplify a bit (assume r is a deterministic function of state)

$$V^\pi(s) = \underbrace{r(s)}_{\text{Receive a reward at state } s} + \gamma \sum_a \underbrace{\pi(a | s)}_{\text{Sample action } a} \sum_{s'} \underbrace{p(s' | s, a)}_{\text{Enter state } s'} \underbrace{V^\pi(s')}_{\text{Expected future rewards}}$$

Policy Evaluation

- Goal: given some π (might not be optimal), compute its value functions

$$V^\pi(s) = \mathbb{E}_\pi[G_0 | s_0 = s]$$

$$Q^\pi(s, a) = \mathbb{E}_\pi[G_0 | s_0 = s, a_0 = a]$$

- Simplify a bit (assume r is only a function of state)

$$V^\pi(s) = \underbrace{r(s)}_{\text{Receive a reward at state } s} + \gamma \sum_a \underbrace{\pi(a | s)}_{\text{Sample action } a} \sum_{s'} \underbrace{p(s' | s, a)}_{\text{Enter state } s'} \underbrace{V^\pi(s')}_{\text{Expected future rewards}}$$

- For Q:

$$Q^\pi(s, a) = \underbrace{r(s)}_{\text{Receive a reward at state } s} + \gamma \sum_{s'} \underbrace{p(s' | s, a)}_{\text{Enter state } s'} \sum_{a'} \underbrace{\pi(a' | s')}_{\text{Sample action } a'} \underbrace{Q^\pi(s', a')}_{\text{Expected future rewards}}$$

Policy Evaluation

$$V^\pi(s) = \underbrace{r(s)}_{\text{Receive a reward at state } s} + \gamma \sum_a \underbrace{\pi(a | s)}_{\text{Sample action } a} \sum_{s'} \underbrace{p(s' | s, a)}_{\text{Enter state } s'} \underbrace{V^\pi(s')}_{\text{Expected future rewards}}$$

❑ For tabular case (grid world), this is a set of linear equations! V^π is its unique solution

❑ Matrix form:

$$V^\pi = r + \gamma T^\pi V^\pi$$

❑ Suppose there are $|S|$ states: $s_1, s_2, \dots$

- T^π is a $|S| \times |S|$ -dim matrix whose (j,k) entry gives $p(s_k | s_j, a = \pi(s_j))$
- V^π is a $|S|$ -dim vector whose (j) entry gives $V^\pi(s_j)$
- r is a $|S|$ -dim vector whose (j) entry gives $r(s_j)$

❑ Then we have a closed-form solution: $V^\pi = (I - \gamma T^\pi)^{-1} r$

❑ Or we can do iterative policy evaluation $V_{k+1} = r + \gamma T^\pi V_k$ until converge

❑ Question: why it can converge? Why is $I - \gamma T^\pi$ invertible?

- It is a **contraction**!

Contraction

□ An operator F on a normed vector space is a γ -contraction ($0 < \gamma < 1$) if:

$$\|F(x) - F(y)\| \leq \gamma \|x - y\|, \forall x, y$$

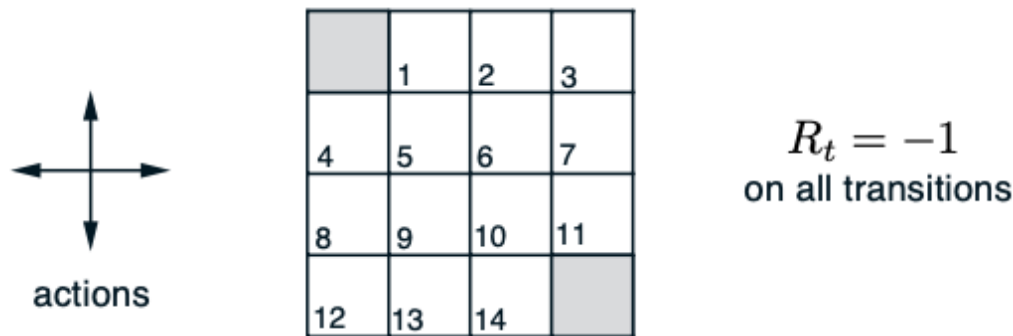
□ If F is a γ -contraction:

- $x_{k+1} = F(x_k)$ converges to a **unique** fixed point in the vector space
- The convergence rate is linear (i.e., exponentially converge)

□ The Bellman operator in policy evaluation is a contraction: $F^\pi(V) = r + \gamma T^\pi V$

$$\begin{aligned}\|F^\pi(u) - F^\pi(v)\|_\infty &= \|(r^\pi + \gamma T^\pi u) - (r^\pi + \gamma T^\pi v)\|_\infty \\ &= \|\gamma T^\pi(u - v)\|_\infty \\ &\leq \|\gamma T^\pi(\mathbf{1}\|u - v\|_\infty)\|_\infty \\ &= \|\gamma(T^\pi \mathbf{1})\|u - v\|_\infty\|_\infty \\ &= \|\gamma \mathbf{1}\|u - v\|_\infty\|_\infty \\ &= \gamma\|u - v\|_\infty\end{aligned}$$

Example: 2-d Grid World



<

Policy Evaluation Summary

- Goal: given some π (might not be optimal), compute its value functions

$$V^\pi(s) = \mathbb{E}_\pi[G_0 | s_0 = s]$$

$$Q^\pi(s, a) = \mathbb{E}_\pi[G_0 | s_0 = s, a_0 = a]$$

$$V^\pi(s) = \underbrace{r(s)}_{\text{Receive a reward at state } s} + \gamma \sum_a \underbrace{\pi(a | s)}_{\text{Sample action } a} \sum_{s'} \underbrace{p(s' | s, a)}_{\text{Enter state } s'} \underbrace{V^\pi(s')}_{\text{Expected future rewards}}$$
$$Q^\pi(s, a) = \underbrace{r(s)}_{\text{Receive a reward at state } s} + \gamma \sum_{s'} \underbrace{p(s' | s, a)}_{\text{Enter state } s'} \sum_{a'} \underbrace{\pi(a' | s')}_{\text{Sample action } a'} \underbrace{Q^\pi(s', a')}_{\text{Expected future rewards}}$$

- Using the above recursive relationship (**Bellman equation**)
- Known as “**dynamic programming**”
- Convergence guarantee because of the contraction!
- Require knowing dynamics

Outline

(Some blackboard today!)

☐ Why RL?

☐ Definitions: MDP, POMDP, return, value functions

Tabular case:

☐ Policy evaluation

☐ Policy improvement

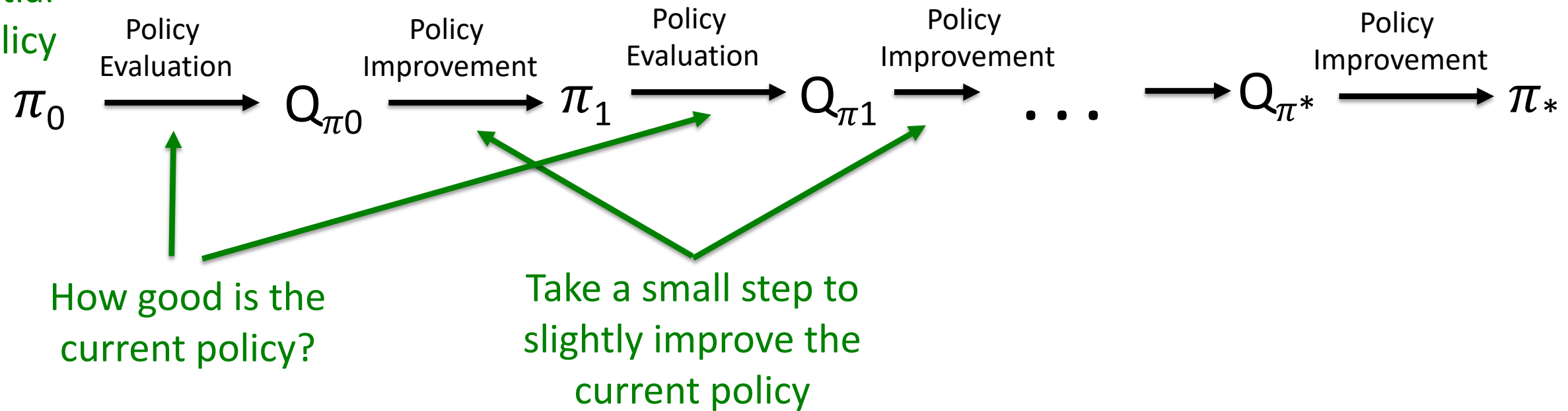
- Policy iteration
- Value iteration

(Some slides are from 22F, 23F, Berkeley CS 285, and CMU 10-703)

Policy Improvement

□ Goal: iteratively improve the policy

Random
initial
policy



What is the long-term
effect of different
actions on the
performance?

Policy Improvement

□ An intuitive idea: $\pi'(s) = \operatorname{argmax}_a Q^\pi(s, a)$

- I.e., “greedily” update policy to optimize previous policy’s Q value

□ Does it work?

□ Yes! **Policy improvement theorem:** if $Q^\pi(s, \pi'(s)) \geq V^\pi(s), \forall s$, then $V^{\pi'}(s) \geq V^\pi(s), \forall s$

□ Proof:

$$\begin{aligned} v_\pi(s) &\leq q_\pi(s, \pi'(s)) \\ &= \mathbb{E}[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s, A_t = \pi'(s)] \\ &= \mathbb{E}_{\pi'}[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s] \\ &\leq \mathbb{E}_{\pi'}[R_{t+1} + \gamma q_\pi(S_{t+1}, \pi'(S_{t+1})) \mid S_t = s] \\ &= \mathbb{E}_{\pi'}[R_{t+1} + \gamma \mathbb{E}[R_{t+2} + \gamma v_\pi(S_{t+2}) \mid S_{t+1}, A_{t+1} = \pi'(S_{t+1})] \mid S_t = s] \\ &= \mathbb{E}_{\pi'}[R_{t+1} + \gamma R_{t+2} + \gamma^2 v_\pi(S_{t+2}) \mid S_t = s] \\ &\leq \mathbb{E}_{\pi'}[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \gamma^3 v_\pi(S_{t+3}) \mid S_t = s] \\ &\vdots \\ &\leq \mathbb{E}_{\pi'}[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \gamma^3 R_{t+4} + \cdots \mid S_t = s] \\ &= v_{\pi'}(s). \end{aligned}$$

Policy Improvement

- ❑ An intuitive idea: $\pi'(s) = \operatorname{argmax}_a Q^\pi(s, a)$
 - I.e., “greedily” update policy to optimize previous policy’s Q value
- ❑ When does $\pi(s) = \operatorname{argmax}_a Q^\pi(s, a)$, policy is unchanged?
- ❑ Bellman optimality equations!

$$V^*(s) = \max_a Q^*(s, a)$$

$$\pi^*(s) = \operatorname{argmax}_a Q^*(s, a)$$

- Using **the principle of optimality**

$$V^*(s) = \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma \cdot V^*(s')]$$

$$Q^*(s, a) = \sum_{s', r} p(s', r | s, a) [r + \gamma \cdot \max_{a'} Q^*(s', a')]$$

Policy Iteration

- Can also do Q value function (policy evaluation is more complex but improvement is easier)

1. Initialization

$V(s) \in \mathbb{R}$ and $\pi(s) \in \mathcal{A}(s)$ arbitrarily for all $s \in \mathcal{S}$

2. Policy Evaluation

Loop:

$\Delta \leftarrow 0$

Loop for each $s \in \mathcal{S}$:

$v \leftarrow V(s)$

$V(s) \leftarrow \sum_{s',r} p(s',r|s,\pi(s)) [r + \gamma V(s')]$

$\Delta \leftarrow \max(\Delta, |v - V(s)|)$

until $\Delta < \theta$ (a small positive number determining the accuracy of estimation)

3. Policy Improvement

policy-stable $\leftarrow$ true

For each $s \in \mathcal{S}$:

old-action $\leftarrow \pi(s)$

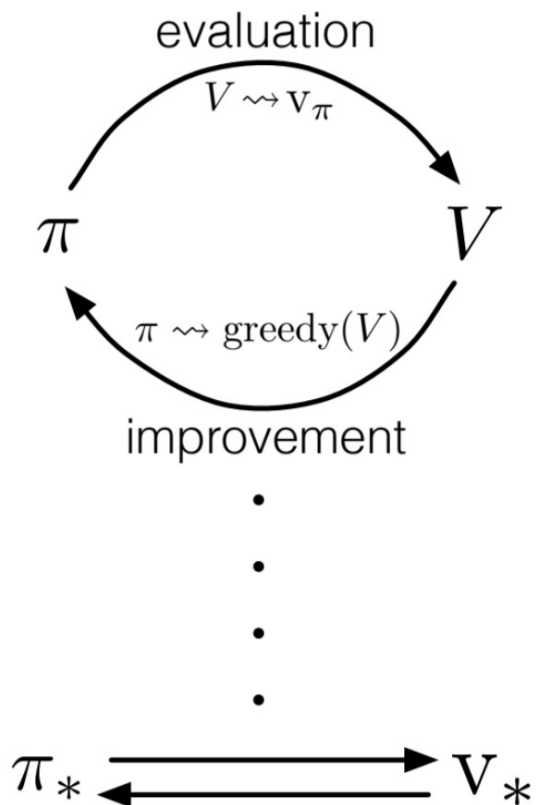
$\pi(s) \leftarrow \operatorname{argmax}_a \sum_{s',r} p(s',r|s,a) [r + \gamma V(s')]$

If *old-action* $\neq \pi(s)$, then *policy-stable* $\leftarrow$ false

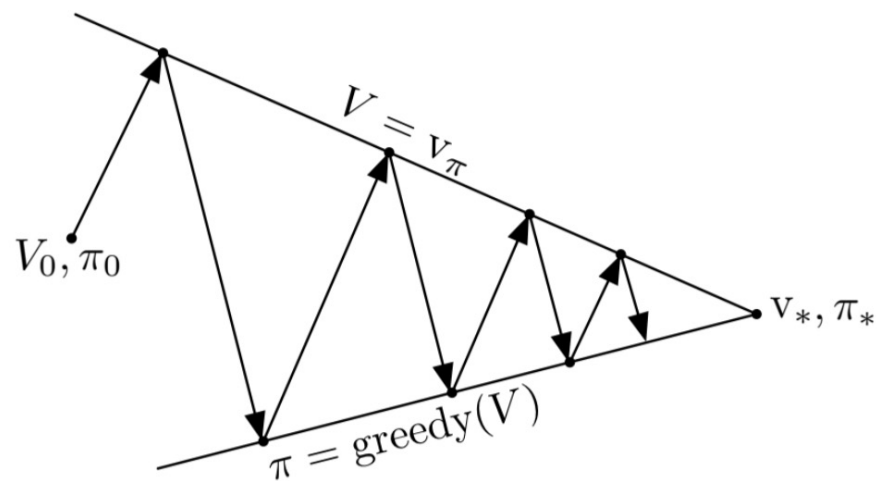
If *policy-stable*, then stop and return $V \approx v_*$ and $\pi \approx \pi_*$; else go to 2

General Policy Iteration

- Any interleaving of policy evaluation and policy improvement
- All RL methods are a form of GPI



A geometric metaphor for convergence of GPI:



Value Iteration

- ❑ Policy iteration involves an iterative policy evaluation step
- ❑ Can we remove it?
- ❑ Yes! Think about Bellman optimality equation:

$$V^*(s) = \max_a Q^*(s, a) = \max_a \sum_{s'} p(s', r | s, a) [r + \gamma \cdot V^*(s')]$$

- ❑ Idea: directly update the value function!

Value Iteration, for estimating $\pi \approx \pi_*$

Algorithm parameter: a small threshold $\theta > 0$ determining accuracy of estimation
Initialize $V(s)$, for all $s \in \mathcal{S}^+$, arbitrarily except that $V(\text{terminal}) = 0$

Loop:

```
|  $\Delta \leftarrow 0$   
| Loop for each  $s \in \mathcal{S}$ :  
|    $v \leftarrow V(s)$   
|    $V(s) \leftarrow \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma V(s')]$   
|    $\Delta \leftarrow \max(\Delta, |v - V(s)|)$   
until  $\Delta < \theta$ 
```

Output a deterministic policy, $\pi \approx \pi_*$, such that

$$\pi(s) = \arg \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma V(s')]$$

Summary

- ❑ Bellman optimality equations are nonlinear
- ❑ In **tabular case with known dynamics**, we can do DP-based policy improvement
- ❑ In general RL settings, **approximately** solve the Bellman optimality equations!

$$V^*(s) = \max_a \sum_{s',r} p(s',r|s,a)[r + \gamma \cdot V^*(s')]$$

$$Q^*(s,a) = \sum_{s',r} p(s',r|s,a)[r + \gamma \cdot \max_{a'} Q^*(s',a')]$$

continuous state/action space (need
function approximation for value
functions)

Unknown dynamics
(must be approximated,
usually using sampling)

max over continuous
action space

Thank You!